Quantile Regression with a Neural Network in PyTorch

1 What Is a Quantile?

Let Y be a random variable with cumulative distribution function $F_Y(y) = P(Y \leq y)$. For a level $q \in (0, 1)$, the *q-th quantile* of Y is the value below which a fraction q of the probability mass lies:

$$Q_Y(q) = \inf\{\, y : F_Y(y) \geq q \,\}.$$

Intuitively, $Q_Y(q)$ answers the question: *"what value is Y below $100q\%$ of the time?"*

- $q = 0.5$ gives the **median**: half the outcomes are below it, half above.
- $q = 0.1$ gives the 10th percentile: only 10% of outcomes fall below it.
- $q = 0.9$ gives the 90th percentile: 90% of outcomes fall below it.

For example, if $Y \sim \mathcal{N}(\mu, \sigma^2)$, then $Q_Y(q) = \mu + z_q\,\sigma$, where z_q is the standard normal quantile ($z_{0.1} \approx -1.28$, $z_{0.5} = 0$, $z_{0.9} \approx 1.28$).

In regression we care about the *conditional* quantile $Q_{Y|X}(q \mid x)$: the q-th quantile of Y given that the input is x. The pair $\big(Q_{Y|X}(0.1 \mid x),\ Q_{Y|X}(0.9 \mid x)\big)$ forms an 80% prediction interval for Y at input x.

2 Linear Regression vs. Quantile Regression

2.1 Linear (least-squares) regression

Ordinary least-squares regression fits a model $\hat{y} = f(x)$ by minimizing the mean squared error:

$$\min_f \ \frac{1}{N} \sum_{i=1}^{N} \big(y_i - f(x_i)\big)^2.$$

The minimizer of the squared error is the **conditional mean** $\mathbb{E}[Y \mid X = x]$. Hence least-squares regression produces a *single number* per input: the average outcome. It says nothing about how spread out the outcomes are around that average.

2.2 Quantile regression

Quantile regression replaces the squared error with the *pinball loss* (Section 3). Its minimizer is the **conditional quantile** $Q_{Y|X}(q \mid x)$. By fitting several quantiles at once we obtain a picture of the whole conditional distribution, not just its centre.

2.3 Key differences

	Least-squares regression	**Quantile regression**
Loss	Squared error $(y - \hat{y})^2$	Pinball loss $L_q(y, \hat{y})$
Estimates	Conditional mean	Conditional q-quantile
Outputs per input	One value	One value per quantile level
Uncertainty	Not captured	Prediction intervals
Outliers	Sensitive (errors squared)	Robust (errors linear)
Noise assumption	Usually constant variance	None; handles changing spread

A note on "linear". "Linear" refers to the form of the model, $f(x) = w^\top x + b$, while "least squares" and "quantile" refer to the loss. Linear quantile regression also exists (a linear model trained with pinball loss). In this project the model is a *neural network*, so it can learn non-linear quantile curves such as $3\sin(x)$.

3 The Pinball Loss

3.1 Definition

Define the residual (error)

$$e = y - \hat{y},$$

where y is the true label and $\hat{y}$ the prediction. A positive e means the model *under-predicted*; a negative e means it *over-predicted*. The pinball loss for quantile level q is

$$L_q(y, \hat{y}) = \begin{cases} q\,(y - \hat{y}) & \text{if } y \geq \hat{y}, \\ (1 - q)\,(\hat{y} - y) & \text{if } y < \hat{y}. \end{cases}$$

3.2 From the formula to a single expression

Substituting $e = y - \hat{y}$, the second case becomes $(1 - q)(-e) = (q - 1)\,e$, so

$$L_q = \begin{cases} q\,e & e \geq 0, \\ (q - 1)\,e & e < 0. \end{cases}$$

Because $0 < q < 1$, the factor q is positive and $q - 1$ is negative. Therefore $q\,e$ and $(q - 1)\,e$ always have opposite signs, and the correct branch is always the non-negative one:

Case	$q\,e$	$(q - 1)\,e$	Maximum
$e \geq 0$ (under-prediction)	≥ 0	≤ 0	$q\,e$
$e < 0$ (over-prediction)	< 0	> 0	$(q - 1)\,e$

This gives the branch-free form used in code:

$$\boxed{L_q(y, \hat{y}) = \max\left(q\,e,\ (q - 1)\,e\right)}$$

3.3 Why it produces quantiles

For $q = 0.9$, under-predicting costs $0.9\,|e|$ while over-predicting costs only $0.1\,|e|$. The model is pushed upward until only about 10% of targets lie above its prediction, which is exactly the 90th percentile. For $q = 0.5$ both sides cost the same, the loss equals $\frac{1}{2}|e|$, and the minimizer is the median.

3.4 Gradient

With respect to the prediction,

$$\frac{\partial L_q}{\partial \hat{y}} = \begin{cases} -q & e > 0, \\ 1 - q & e < 0. \end{cases}$$

The loss is not differentiable at $e = 0$; PyTorch's `torch.max` simply picks one branch there, which works fine in practice. Note that the gradient does not shrink as $e \to 0$ (unlike squared error), which is why pinball loss tends to need more training epochs.

4 Implementation, Step by Step

The notebook is described cell by cell, in execution order.

4.1 Step 1: Imports

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import TensorDataset, DataLoader
import matplotlib.pyplot as plt
```

- `torch`: tensors and automatic differentiation.
- `torch.nn`: neural-network layers.
- `torch.optim`: optimizers such as Adam.
- `TensorDataset`, `DataLoader`: wrap tensors and serve them in shuffled mini-batches.
- `matplotlib.pyplot`: plotting.

4.2 Step 2: Choosing the device

```
device = "cuda" if torch.cuda.is_available() else "cpu"
print(device)
```

Use the GPU if one is available, otherwise the CPU. All model parameters and every tensor fed to the model must live on the same device.

4.3 Step 3: Generating synthetic data

```
n = 10000
x = torch.rand(n, 1) * 10
noise = torch.randn(n, 1) * (0.1 + 0.3 * x)
y = (torch.sin(x) * 3 + noise).squeeze(1)
loader = DataLoader(TensorDataset(x, y), batch_size=128, shuffle=True)
```

The data are generated from

$$y = 3\sin(x) + \varepsilon, \qquad \varepsilon \sim \mathcal{N}\big(0,\ (0.1 + 0.3x)^2\big), \qquad x \sim \mathcal{U}(0, 10).$$

- `x` has shape $(n, 1)$: n samples, one feature each.
- The noise standard deviation grows with x (*heteroscedastic* noise). This is where quantile regression shines: least squares would return only the curve $3\sin(x)$, while the quantile bands widen as x grows.
- `.squeeze(1)` removes the size-1 column, so `y` has shape $(n,)$.
- Because the generating process is known, so are the true quantiles: $Q(q \mid x) = 3\sin(x) + z_q\,(0.1 + 0.3x)$. The trained model can be checked against them.
- The `DataLoader` shuffles the data each epoch and yields batches of 128 samples.

4.4 Step 4: The neural network

```
class NeuralNetwork(nn.Module):
    def __init__(self, in_dim, hid_dim, out_dim):
        super().__init__()
        self.forward_pass = nn.Sequential(
            nn.Linear(in_dim, hid_dim),
            nn.ReLU(),
            nn.Linear(hid_dim, hid_dim),
            nn.ReLU(),
            nn.Linear(hid_dim, out_dim),
        )

    def forward(self, x):
        return self.forward_pass(x)
```

- `super().__init__()` initializes the parent `nn.Module`, which is what lets PyTorch track the layers' parameters. It takes no arguments: `super()` already binds `self`.
- The network is a multilayer perceptron: two hidden layers of width `hid_dim` with ReLU activations, mapping $\mathbb{R}^{\text{in_dim}} \to \mathbb{R}^{\text{out_dim}}$.
- `out_dim` equals the number of quantiles. Each output neuron predicts one quantile, so a single network learns all of them at once and they share the hidden features.

4.5 Step 5: The pinball loss

```
def pinball_loss(prediction, label, quantiles):
    error = label.view(-1, 1) - prediction
    return torch.max(quantiles * error, (quantiles - 1) * error).mean()
```

This is the boxed formula from Section 3, applied to a whole batch at once. The shapes are:

- `label`: $(N,)$, the true values y.
- `prediction`: (N, Q), the predictions $\hat{y}$ for Q quantiles.
- `quantiles`: $(Q,)$, the levels $q_1, \ldots, q_Q$.

Reshaping with `view`. `view` reshapes a tensor without copying its data. In `view(-1, 1)`, the `1` requests exactly one column and the `-1` tells PyTorch to infer the number of rows from the total size. The label therefore becomes a column of shape $(N, 1)$.

Broadcasting. Subtracting (N, Q) from $(N, 1)$ stretches the single column across all Q columns:

$$\texttt{error}_{ij} = y_i - \hat{y}_{ij},$$

so every sample's label is compared against each of its Q quantile predictions. Similarly, `quantiles` of shape $(Q,)$ is broadcast across rows, so column j is weighted by its own q_j. Without the `view`, an $(N,)$ label would be aligned with the last dimension Q, which is wrong.

Reduction. `.mean()` with no arguments averages over all $N \times Q$ entries and returns a 0-dimensional (scalar) tensor:

$$\mathcal{L} = \frac{1}{NQ} \sum_{i=1}^{N} \sum_{j=1}^{Q} \max\left(q_j\, e_{ij},\ (q_j - 1)\, e_{ij}\right).$$

A scalar is what `loss.backward()` requires. Note that `.mean()` returns a scalar even if the shapes are wrong (e.g. an $(N,)$ prediction broadcasts to (N, N)), so a shape mistake fails silently rather than with an error.

4.6 Step 6: Hyperparameters

```
quantiles = torch.tensor([0.1, 0.5, 0.9], device=device)
in_dim = 1
hid_dim = 64
out_dim = len(quantiles)
lr = 1e-3
batch_size = 128
epochs = 500
```

- Quantiles 0.1, 0.5, 0.9: a lower bound, the median, and an upper bound, forming an 80% prediction interval. The tensor is placed on `device` because it is multiplied with GPU tensors in the loss.
- `in_dim = 1`: one input feature.

- `hid_dim = 64`: ample capacity for a one-dimensional curve.
- `lr = 1e-3`: the standard Adam learning rate.
- `epochs = 500`: pinball loss converges more slowly than squared error because its gradient does not shrink near the optimum.

4.7 Step 7: Training

```
model = NeuralNetwork(in_dim, hid_dim, out_dim).to(device)
optimizer = optim.Adam(model.parameters(), lr)

for epoch in range(epochs):
    for xb, yb in loader:
        xb, yb = xb.to(device), yb.to(device)
        optimizer.zero_grad()
        loss = pinball_loss(model(xb), yb, quantiles)
        loss.backward()
        optimizer.step()
    if epoch % 50 == 0:
        print(epoch)
```

Each iteration over a mini-batch performs:

1. **Move to device**: copy the batch to the GPU.
2. **Zero gradients**: PyTorch accumulates gradients by default, so the previous step's gradients must be cleared.
3. **Forward pass**: compute predictions `model(xb)` and the scalar loss.
4. **Backward pass**: `loss.backward()` uses automatic differentiation to compute $\partial \mathcal{L}/\partial \theta$ for every parameter θ. No hand-written backward pass is needed.
5. **Update**: `optimizer.step()` applies the Adam update to the parameters.

Naming pitfall. The batch variables are deliberately named `xb`, `yb` and not `x`, `y`. Python has no block scope: a `for` loop assigns to its variables in the enclosing scope, which in a notebook is the global scope. Writing `for x, y in loader` would overwrite the full dataset with the last mini-batch, which is a GPU tensor.

4.8 Step 8: Visualizing the result

```
x, y = loader.dataset.tensors
xs = torch.linspace(0, 10, 200).view(-1, 1)
with torch.no_grad():
    preds = model(xs.to(device)).cpu()

plt.scatter(x, y, s=2, alpha=0.3)
for i, q in enumerate([0.1, 0.5, 0.9]):
    plt.plot(xs, preds[:, i], label=f"q={q}")
plt.legend()
plt.savefig("quantiles.pdf", bbox_inches="tight")
```

- `loader.dataset.tensors` recovers the exact (CPU) training data.
- `xs` is a dense grid of 200 points, reshaped to $(200, 1)$ to match the model's input shape.
- `torch.no_grad()` disables gradient tracking, since we are only predicting.
- `.cpu()` moves the predictions back to the CPU: matplotlib converts tensors to NumPy arrays, which is only possible for CPU tensors.
- Column i of `preds` is the curve for quantile q_i.

What to expect. The $q = 0.5$ curve should follow $3\sin(x)$, and the $q = 0.1$ and $q = 0.9$ curves should form a band that widens as x increases, mirroring the growing noise. About 10% of points should lie below the lower curve and about 10% above the upper one. If two quantile curves cross somewhere, train longer or sort the outputs per input with `preds.sort(dim=1).values`. Figure 1 shows the result.

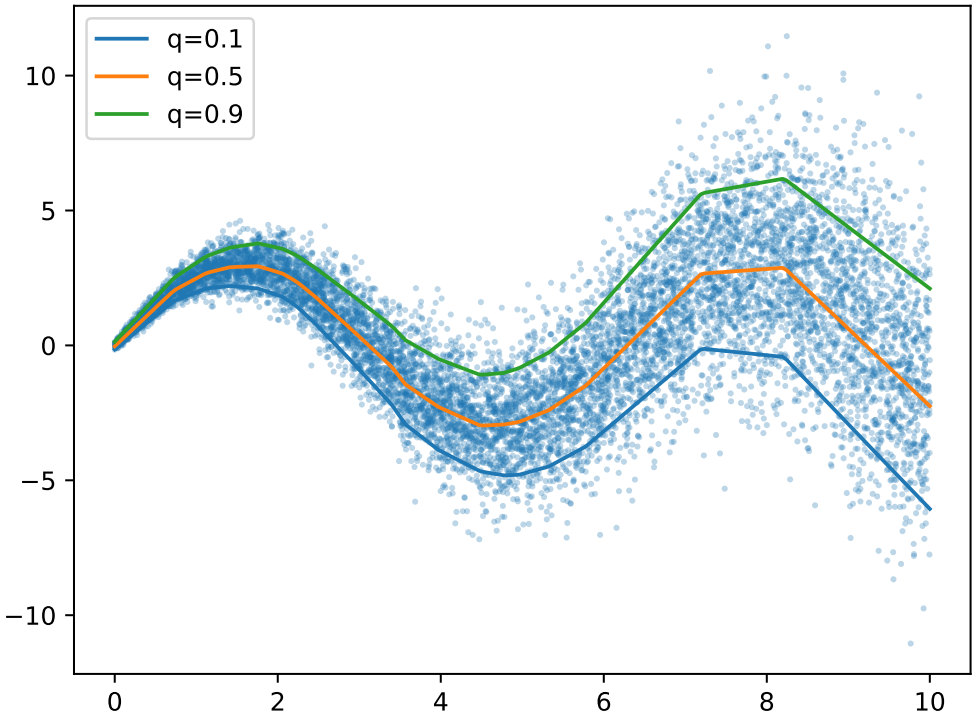


Figure 1: Training data (blue dots) and the learned conditional quantile curves for $q = 0.1$, 0.5 and 0.9.

5 Summary

1. A quantile $Q(q)$ is the value below which a fraction q of outcomes fall.
2. Least-squares regression estimates the conditional mean; quantile regression estimates conditional quantiles and hence the spread of the outcomes.

3. The pinball loss $\max(q\,e, (q-1)\,e)$ penalizes under- and over-prediction asymmetrically, and its minimizer is the q-quantile.

4. A single neural network with one output per quantile, trained with this loss via autograd and Adam, learns non-linear quantile curves and so captures noise whose spread changes with the input.